Unit 7

Unit 7: Introduction to NoSQL and MongoDB

1. Overview of NoSQL

NoSQL stands for **"Not Only SQL"**. It is a category of database management systems that do not follow the traditional relational (table-based) model. NoSQL databases were developed in response to the limitations of relational databases when dealing with very large volumes of data, high-speed data ingestion, and data that does not fit neatly into rows and columns.

The term "Not Only SQL" reflects that these systems do not replace SQL entirely — rather, they offer an alternative or complement to relational databases depending on the use case.

NoSQL databases became prominent with the rise of big data applications, social media platforms, real-time web applications, and cloud computing — scenarios where relational databases struggled to scale efficiently.

Why did NoSQL emerge?

Relational databases were designed in an era when data was structured, relatively small in volume, and consistency was the top priority. Modern applications changed these requirements:

- Social media platforms like Facebook and Twitter generate billions of records daily — relational databases couldn't scale horizontally (across many servers) efficiently.
- Data from the web is often unstructured or semi-structured (JSON, XML, logs, multimedia).
- Applications need to read and write data at very high speed with low latency.
- Data schemas in modern applications change frequently — relational schemas are rigid and hard to modify.

NoSQL was built to address exactly these problems.

---

2. Characteristics of NoSQL

NoSQL databases share the following common characteristics:

2.1 Schema-less (Flexible Schema)

NoSQL databases do not require a fixed, predefined schema. Different records in the same collection can have different fields. This is extremely useful when the structure of data is not known in advance or changes over time. In relational databases, adding a new column requires altering the entire table and

dealing with NULL values for existing rows — in NoSQL, you simply start storing the new field in new documents.

2.2 Horizontal Scalability

Relational databases scale **vertically** — by upgrading hardware (more CPU, more RAM) on a single server. This has limits and is expensive. NoSQL databases are designed to scale **horizontally** — by adding more servers (nodes) to a cluster. Data is distributed across multiple machines. This is how companies like Google, Amazon, and Facebook handle billions of records.

2.3 High Performance

Because NoSQL databases avoid the overhead of joins, foreign key enforcement, and ACID transaction management (in many cases), they can read and write data much faster than relational databases — especially at scale.

2.4 High Availability and Fault Tolerance

Most NoSQL databases are designed with **replication** built in. Data is automatically copied across multiple nodes. If one node fails, another takes over — there is no single point of failure. The system continues to operate even during hardware failures.

2.5 Distributed Architecture

NoSQL databases are built from the ground up to run across distributed systems — many machines working together. Data is partitioned (sharded) across nodes, allowing the system to handle data sets that are too large for a single machine.

2.6 BASE Properties (instead of ACID)

Relational databases follow **ACID** (Atomicity, Consistency, Isolation, Durability). NoSQL databases typically follow **BASE**:

- **B**asically **A**vailable — the system guarantees availability
- **S**oft state — the state of the system may change over time even without input
- **E**ventually consistent — the system will become consistent over time, but not necessarily immediately

This trade-off of strict consistency for availability and performance is a defining characteristic of NoSQL.

2.8 CAP Theorem

The **CAP Theorem** (also called Brewer's Theorem, proposed by Eric Brewer in 2000) states that a distributed database system can only guarantee **two out of the following three properties** at the same time — never all three simultaneously.

The Three Properties

C — Consistency
Every read receives the most recent write or an error. In other words, all nodes in the distributed system see the same data at the same time. If you write a value to one node, any subsequent read from any other node will return that updated value. There is no possibility of reading stale/old data.

A — Availability
Every request (read or write) receives a response — it will not time out or return an error, even if some nodes in the system are down. The system is always operational and always responds, though the response may not be the most up-to-date data.

P — Partition Tolerance
The system continues to function even if communication between some nodes in the distributed cluster breaks down (a network partition). Messages between nodes may be delayed or lost, but the system keeps running.

Why Can't You Have All Three?

In a distributed system, **network partitions are inevitable** — network failures, packet loss, and delays will happen. So in practice, Partition Tolerance (P) is not optional — every distributed system must tolerate partitions to be useful. This means the real trade-off is always between **Consistency (C)** and **Availability (A)** when a partition occurs.

When a partition happens:

- If you choose **Consistency** — the system will refuse to respond until all nodes are synchronized. Some requests may fail or time out. The data is always correct but the system may be temporarily unavailable.
- If you choose **Availability** — the system always responds, but different nodes may return different (possibly stale) values until they synchronize. The system is always up but data may be temporarily inconsistent.

The Three CAP Combinations

Type	Guarantees	Trade-off	Examples
CP (Consistent + Partition Tolerant)	Data is always consistent across nodes	May be unavailable during a partition	MongoDB, HBase, Redis, Zookeeper
AP (Available + Partition Tolerant)	System always responds	Data may be temporarily inconsistent (eventual consistency)	Cassandra, CouchDB, DynamoDB
CA (Consistent + Available)	Data is consistent and system always responds	Cannot tolerate network partitions — only practical on a single node	Traditional RDBMS (MySQL, PostgreSQL) — but these are not truly distributed

> **Note:** CA systems are essentially non-distributed systems. In a real distributed environment, you must tolerate partitions, so CA is not a realistic option for distributed NoSQL databases.

Relationship with BASE

The CAP theorem directly explains why NoSQL databases use **BASE** instead of **ACID**. AP systems that choose availability over consistency during partitions cannot guarantee immediate consistency — which is exactly what BASE's "Eventually Consistent" captures. ACID's strict consistency guarantee is essentially the C in CAP, which AP systems trade away for availability.

```
ACID  →  Prioritizes Consistency (C in CAP)
BASE  →  Prioritizes Availability (A in CAP) with eventual consistency
```

2.8 Support for Unstructured and Semi-structured Data

NoSQL databases natively store data in formats like JSON, XML, key-value pairs, graphs, and binary files — all of which are difficult to represent in flat relational tables.

---

3. Storage Types of NoSQL

NoSQL databases are not a single type — they are a family of different storage models, each suited for different kinds of problems.

3.1 Key-Value Stores

This is the simplest NoSQL model. Data is stored as **key-value pairs**, similar to a dictionary or a hash map. You look up data by its key, and the value can be anything — a string, a number, a JSON object, a binary blob.

- **Structure:** `key → value`
- **Best for:** Caching, session management, user preferences, shopping carts
- **Examples:** Redis, Amazon DynamoDB, Riak

Example:

```
"user:1001" → { name: "Alice", age: 22 }
"user:1002" → { name: "Bob", age: 25 }
```

3.2 Document Stores

Data is stored as **documents**, typically in JSON or BSON (Binary JSON) format. Each document is a self-contained unit that can have nested fields, arrays, and sub-documents. Documents in the same collection do not need to have the same structure.

- **Structure:** Collections of JSON/BSON documents
- **Best for:** Content management systems, catalogs, user profiles, e-commerce

- **Examples:** MongoDB, CouchDB, Firestore

Example:

```
{
  "_id": "001",
  "name": "Alice",
  "courses": ["DBMS", "OS", "Networks"],
  "address": {
    "city": "Mumbai",
    "pin": "400001"
  }
}
```

3.3 Column-Family Stores (Wide-Column Stores)

Data is stored in tables with rows and dynamic columns, but unlike relational tables, different rows can have completely different columns. Data is stored column by column rather than row by row, which makes reading a specific column across many rows very fast.

- **Structure:** Rows identified by a key, with flexible columns grouped into column families
- **Best for:** Analytics, time-series data, IoT sensor data, logging systems
- **Examples:** Apache Cassandra, HBase, Google Bigtable

3.4 Graph Databases

Data is stored as **nodes** (entities) and **edges** (relationships between entities), with properties on both. Graph databases are optimized for queries that traverse relationships — something relational databases handle very poorly with multiple self-joins.

- **Structure:** Nodes, Edges, Properties
- **Best for:** Social networks, recommendation engines, fraud detection, network topology
- **Examples:** Neo4j, Amazon Neptune, ArangoDB

Example: In a social network, each person is a node. "Follows", "Friends with", and "Likes" are edges between nodes.

3.5 Time Series Databases

A time series database is purpose-built for storing and querying **data points that are indexed by time**. Every record has a timestamp, and the primary access pattern is always time-based — you query data for a specific time range, look for trends over time, or compute aggregates (average, min, max) over time windows.

Unlike general-purpose NoSQL databases, time series databases are heavily optimized for:

- **High-speed sequential writes** — data arrives continuously (every second, every millisecond) and must be written very fast
- **Time-range queries** — "give me all readings between 9 AM and 10 AM"
- **Automatic data retention policies** — old data is often downsampled or deleted after a set period to save space
- **Compression** — since time-series data is highly repetitive (values close in time are often similar), specialized compression algorithms reduce storage dramatically
- **Structure:** Measurements indexed by timestamp, with optional tags/labels for filtering
- **Best for:** IoT sensor data, application performance monitoring, financial tick data, server metrics, weather data
- **Examples:** InfluxDB, TimescaleDB, OpenTSDB, Prometheus

Example:

timestamp	sensor_id	temperature	humidity
2024-01-15 09:00:00	S001	24.5	60
2024-01-15 09:00:01	S001	24.6	61
2024-01-15 09:00:02	S001	24.5	60

Every row is a measurement at a point in time. The database is optimized to answer questions like: "What was the average temperature for sensor S001 between 9 AM and 10 AM?" extremely fast.

---

4. SQL vs NoSQL

Feature	SQL (Relational)	NoSQL
Data Model	Tables with rows and columns	Key-value, Document, Column, Graph
Schema	Fixed, predefined schema	Flexible / schema-less
Scalability	Vertical (scale up)	Horizontal (scale out)
Query Language	SQL (standardized)	Varies by database (no standard)
Joins	Supports complex joins	Generally no joins
Transactions	Full ACID compliance	BASE (eventual consistency, usually)
Data Type	Structured data only	Structured, semi-structured, unstructured
Relationships	Enforced via foreign keys	Handled in application logic

Feature	SQL (Relational)	NoSQL
Best For	Banking, ERP, accounting, structured data	Big data, real-time apps, social media, IoT
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra, Neo4j

When to use SQL:

- Data is structured and relationships between entities are well-defined
- Data integrity and consistency are critical (e.g., financial systems)
- Complex queries with multiple joins are needed
- The schema is unlikely to change frequently

When to use NoSQL:

- Data volume is very large and needs to scale across many servers
- Data is unstructured, semi-structured, or changes shape frequently
- High write/read throughput is needed
- Schema flexibility is important (rapid prototyping, evolving data models)

---

5. What is MongoDB?

MongoDB is an open-source, document-oriented NoSQL database. It was developed by **MongoDB Inc.** (originally 10gen) and first released in 2009. The name comes from the word "humongous" — reflecting its ability to handle huge volumes of data.

MongoDB stores data as **BSON documents** (Binary JSON), which are rich, flexible JSON-like structures that can contain nested fields and arrays. These documents are organized into **collections**, which are analogous to tables in a relational database.

MongoDB is designed to be:

- **Scalable** — supports horizontal scaling via sharding
- **Flexible** — no fixed schema required
- **Developer-friendly** — data is stored in a format that maps naturally to objects in most programming languages (JSON ↔ Python dict, JavaScript object, Java Map, etc.)
- **High-performance** — supports indexing, in-memory processing, and native aggregation

MongoDB is the most popular NoSQL database in the world and is widely used in web applications, mobile backends, analytics platforms, and real-time data systems.

---

6. Key Characteristics of MongoDB

6.1 Document-Oriented Storage

MongoDB stores data as BSON (Binary JSON) documents. A document is a set of key-value pairs, similar to a JSON object. Documents can contain arrays, nested objects, and a wide variety of data types. Each document is a self-contained unit — you don't need to join documents to get all the data about one entity.

6.2 Dynamic Schema (Schema-less)

In MongoDB, documents in the same collection are not required to have the same fields or data types. You can add new fields to some documents without affecting others. This is ideal for applications where the data model evolves over time.

6.3 Indexing

MongoDB supports indexing on any field in a document, including fields inside nested sub-documents and arrays. Indexes speed up query performance significantly. MongoDB also supports compound indexes, text indexes (for full-text search), and geospatial indexes.

6.4 Replication (Replica Sets)

MongoDB uses **replica sets** to provide high availability. A replica set is a group of MongoDB servers that maintain the same data. One node is the **primary** (handles all writes), and the others are **secondaries** (replicate data from the primary). If the primary fails, one of the secondaries is automatically elected as the new primary — ensuring the system stays online.

6.5 Sharding (Horizontal Scaling)

When a dataset grows too large for a single server, MongoDB can **shard** the data — splitting it across multiple servers (shards). Each shard holds a subset of the data. MongoDB handles routing queries to the correct shard transparently, so applications do not need to know how data is distributed.

6.6 Aggregation Framework

MongoDB has a powerful built-in **aggregation pipeline** that allows complex data transformations and analysis — similar to GROUP BY, JOIN, and aggregate functions in SQL — all within the database itself.

6.7 GridFS

For storing and retrieving files larger than 16 MB (MongoDB's document size limit), MongoDB provides **GridFS** — a specification for storing large files by splitting them into smaller chunks and storing each chunk as a separate document.

6.8 Ad Hoc Queries

MongoDB supports rich, expressive queries on any field in a document at any time — you are not restricted to pre-defined query structures.

---

7. MongoDB vs SQL — Key Mapping

When people first move from SQL to MongoDB, the biggest confusion is terminology. The underlying concepts are the same — only the names change. Here is the full mapping with a description of each:

SQL Term	MongoDB Term	Description
Database	Database	A container that holds all related collections. Works the same way in both — `use college` in MongoDB switches to the college database, creating it if it doesn't exist yet.
Table	Collection	A group of related documents, just like a table is a group of related rows. The key difference: a collection has **no enforced schema** — documents inside it can have completely different fields from one another.
Row / Record	Document	One entry of data. In SQL a row is flat — just a fixed set of column values. In MongoDB a document is a JSON/BSON object that can contain nested sub-documents and arrays, giving it a much richer, hierarchical structure.
Column	Field	A single attribute or property of a record. In SQL every row has the same columns. In MongoDB different documents in the same collection can have entirely different fields — one document can have 3 fields and another in the same collection can have 10.
Primary Key	`_id` field	A unique identifier for each record. In SQL you define this manually and choose the column. In MongoDB every document automatically gets an `_id` field — if you don't provide one at insert time, MongoDB generates a unique **ObjectId** for it automatically. It cannot be null and must be unique within the collection. You can never have two documents with the same `_id` in a collection.
Foreign Key	Reference / Embedded Document	In SQL a foreign key explicitly links two tables and the database enforces the relationship (you cannot insert a child row whose foreign key has no matching parent). In MongoDB relationships are handled either by **embedding** (putting the related data directly inside the parent document) or by **referencing** (storing the `_id` of the related document as a field). MongoDB does **not** enforce this at the database level — referential integrity is managed in application logic.
JOIN	`$lookup` / Embedding	SQL JOINs combine rows from multiple tables at query time into a single result. MongoDB avoids JOINs by design — related data is usually embedded in a single document so everything is retrieved in one read operation. When you do need to combine data from two collections, MongoDB's aggregation pipeline provides the `$lookup` stage, which works similarly to a SQL LEFT JOIN.
Index	Index	Both SQL and MongoDB support indexes on columns/fields to speed up query performance. MongoDB additionally supports indexes on nested fields (e.g., `"address.city"`), array fields, text fields (for full-text search), and geospatial data.
Schema	Dynamic Schema	In SQL the schema is fixed — you define all columns when you CREATE TABLE, and every row must follow that structure. If you

SQL Term	MongoDB Term	Description
		want to add a column later, you run ALTER TABLE, which can be expensive on large tables. In MongoDB there is no enforced schema — you can add, remove, or change fields in documents at any time without any migration step.
View	View	Both support views — stored query definitions that behave like virtual tables/collections. In MongoDB a view is created using the aggregation pipeline and is read-only.

---

8. MongoDB Data Models — Embedded vs Normalized

When designing a MongoDB schema, one of the most important decisions is how to represent **relationships between data**. Unlike SQL where all relationships are handled via foreign keys and joins, MongoDB gives you two explicit choices:

8.1 Embedded Data Model (Denormalized)

In the embedded model, related data is **stored directly inside the parent document** as a nested sub-document or array. Instead of storing the relationship across two separate collections, everything about an entity is contained in a single document.

Example — a student and their address:

```
{
  "_id": "S001",
  "name": "Alice",
  "branch": "CS",
  "address": {
    "street": "12 MG Road",
    "city": "Mumbai",
    "pin": "400001"
  }
}
```

The address is not in a separate `Address` collection — it lives inside the student document itself.

Advantages:

- A single read operation fetches all related data — no joins needed
- Faster read performance
- Data that belongs together is stored together

Disadvantages:

- If the same data needs to be embedded in many documents, it gets duplicated across all of them — redundancy
- If the embedded data changes, you must update every document that contains it

---

One-to-One Relationship Using Embedding

A one-to-one relationship means one document is associated with exactly one other document. This is the most natural fit for embedding — since there is only one related item, it adds no overhead to embed it directly.

Example: One student has one passport.

```
{
  "_id": "S001",
  "name": "Alice",
  "passport": {
    "passport_no": "A1234567",
    "expiry": "2030-05-01",
    "country": "India"
  }
}
```

The passport details are embedded directly inside the student document. There is no separate `Passport` collection. When you retrieve Alice's document, you automatically get her passport data too — one read, all data.

One-to-Many Relationship Using Embedding

A one-to-many relationship means one document is associated with multiple related items. If the "many" side is a small, bounded set of items (not growing indefinitely) and those items are always accessed together with the parent, embedding works well.

Example: One student has many phone numbers.

```
{
  "_id": "S001",
  "name": "Alice",
  "phones": [
    { "type": "mobile", "number": "9876543210" },
    { "type": "home",   "number": "02222334455" }
  ]
}
```

Another Example: One blog post has many comments.

```
{
  "_id": "P001",
  "title": "MongoDB Basics",
  "author": "Alice",
  "comments": [
    { "user": "Bob",   "text": "Great post!",    "date": "2024-01-10" },
    { "user": "Carol", "text": "Very helpful.",  "date": "2024-01-11" }
  ]
}
```

8.2 Normalized Data Model (Referenced)

In the normalized model, related data is stored in **separate collections**, and documents reference each other using the `_id` field — similar to foreign keys in SQL. Instead of embedding, you store only the `_id` of the related document and perform a separate query (or use `$lookup`) to fetch it when needed.

Example — a student referencing a department:

```
// students collection
{
  "_id": "S001",
  "name": "Alice",
  "dept_id": "D01"
}

// departments collection
{
  "_id": "D01",
  "dept_name": "Computer Science",
  "hod": "Dr. Sharma",
  "budget": 500000
}
```

Here `dept_id` in the student document is a reference to the `_id` in the departments collection — exactly like a foreign key in SQL.

Advantages:

- No data duplication — department data is stored only once
- Updating department information requires changing only one document
- Works well when the related data is large, frequently updated, or shared across many documents

Disadvantages:

- Requires multiple read operations (or a `$lookup` aggregation) to assemble the full data

- Slightly more complex queries

8.3 When to Use Each Model

Situation	Use Embedded	Use Normalized (Referenced)
Data is always accessed together	✓	
One-to-one relationship	✓	
One-to-many, "many" side is small and bounded	✓	
Related data changes rarely	✓	
Data is shared across many documents		✓
The "many" side can grow very large (unbounded)		✓
Related data is frequently updated		✓
Many-to-many relationship		✓
You need to query the related data independently		✓

Rule of thumb:

> If you find yourself always retrieving two pieces of data together — embed them. If the related data is large, shared by multiple parent documents, or needs to be queried on its own — normalize (reference) them.

---

9. MongoDB Data Types

MongoDB's BSON format supports a richer set of data types than plain JSON.

Data Type	Description	Example
String	UTF-8 encoded text	`"Alice"`
Integer	32-bit or 64-bit whole number	`22`, `1000000`
Double	64-bit floating point number	`3.14`
Boolean	True or False	`true`, `false`
Date	Stores date and time in milliseconds since Unix epoch	`ISODate("2024-01-15")`
ObjectId	A 12-byte unique identifier, auto-generated for `_id`	`ObjectId("507f1f...")`

Data Type	Description	Example
Array	An ordered list of values (can be of mixed types)	`["Math", "Science"]`
Embedded Document	A nested document (sub-document) inside a document	`{ city: "Mumbai", pin: "400001" }`
Null	Represents a null or missing value	`null`
Regular Expression	BSON supports storing regex patterns	`/^alice/i`
Binary Data	Stores binary content (images, files)	Binary blob
JavaScript	Stores JavaScript code	`function() {}`

The `_id` Field

Every MongoDB document automatically has an `_id` field. If you do not provide one during insertion, MongoDB generates a unique **ObjectId** for it automatically. The `_id` field acts as the primary key for the document within its collection — it must be unique and cannot be null.

---

10. Managing Databases and Collections from the MongoDB Shell

The **MongoDB Shell** (`mongosh`) is an interactive JavaScript interface to MongoDB. You use it to connect to a MongoDB server, run commands, and manage databases and collections.

10.1 Starting the Shell

```
mongosh
```

This connects to the default MongoDB instance running on `localhost:27017`.

10.2 Database Commands

Show all databases:

```
show dbs
```

Switch to (or create) a database:

```
use college
```

If the database `college` does not exist, MongoDB creates it as soon as you insert data into it.

Show the current database:

```
db
```

Drop (delete) the current database:

```
db.dropDatabase()
```

10.3 Collection Commands

Show all collections in the current database:

```
show collections
```

Create a collection explicitly:

```
db.createCollection("students")
```

Collections can also be created implicitly — the moment you insert a document into a collection that doesn't exist, MongoDB creates it automatically.

Drop (delete) a collection:

```
db.students.drop()
```

Rename a collection:

```
db.students.renameCollection("learners")
```

---

11. MongoDB Basic Commands — CRUD Operations

CRUD stands for **Create, Read, Update, Delete** — the four fundamental operations performed on any database.

11.1 CREATE — Inserting Documents

Insert a Single Document — `insertOne()`

Inserts exactly one document into the collection.

```
db.students.insertOne({
    name: "Alice",
    age: 21,
    branch: "CS",
    year: 2,
    email: "alice@college.com"
})
```

MongoDB automatically adds an `_id` field if not provided. The output confirms the insertion with `acknowledged: true` and the generated `insertedId`.

Insert Multiple Documents — `insertMany()`

Inserts an array of documents in a single operation.

```
db.students.insertMany([
    { name: "Bob",   age: 22, branch: "IT",  year: 3 },
    { name: "Carol", age: 20, branch: "CS",  year: 1 },
    { name: "Dave",  age: 23, branch: "ECE", year: 4 }
])
```

The output includes `insertedCount` and an array of `insertedIds`.

11.2 READ — Finding Documents

Find All Documents — `find()`

Returns all documents in the collection.

```
db.students.find()
```

Find with a Filter

Pass a query object to filter documents. Only documents matching the condition are returned.

```
// All students in the CS branch
db.students.find({ branch: "CS" })

// Students in year 2
db.students.find({ year: 2 })

// Students in CS branch AND in year 2
db.students.find({ branch: "CS", year: 2 })
```

Find One Document — `findOne()`

Returns the **first** document that matches the filter.

```
db.students.findOne({ name: "Alice" })
```

Projection — Selecting Specific Fields

The second argument to `find()` specifies which fields to include (1) or exclude (0) in the output.

```
// Show only name and branch; hide _id
db.students.find({}, { name: 1, branch: 1, _id: 0 })

// Show all fields except email
db.students.find({}, { email: 0 })
```

Counting Documents

```
db.students.countDocuments({ branch: "CS" })
```

11.3 UPDATE — Modifying Documents

Update operations use **update operators** — special keywords prefixed with `$` that specify how to modify the document.

Operator	Purpose
`$set`	Set the value of a field (adds the field if it doesn't exist)
`$unset`	Remove a field from the document
`$inc`	Increment a numeric field by a given amount
`$push`	Add an element to an array field
`$pull`	Remove an element from an array field

Update a Single Document — `updateOne()`

Updates the **first** document matching the filter.

```
// Set Alice's year to 3
db.students.updateOne(
    { name: "Alice" },
    { $set: { year: 3 } }
)

// Add a new field 'phone' to Alice's document
db.students.updateOne(
    { name: "Alice" },
    { $set: { phone: "9876543210" } }
)

// Increment Alice's age by 1
db.students.updateOne(
    { name: "Alice" },
    { $inc: { age: 1 } }
)
```

Update Multiple Documents — `updateMany()`

Updates **all** documents matching the filter.

```
// Set all CS students to year 2
db.students.updateMany(
    { branch: "CS" },
    { $set: { year: 2 } }
)
```

Replace a Document — `replaceOne()`

Replaces the **entire** document (except `_id`) with a new document.

```
db.students.replaceOne(
    { name: "Bob" },
    { name: "Bob", age: 23, branch: "IT", year: 4, email: "bob@college.com" }
)
```

> **Note:** `$set` modifies only specified fields; `replaceOne()` replaces the whole document. Use `$set` when you want to update specific fields without touching the rest.

11.4 DELETE — Removing Documents

Delete a Single Document — `deleteOne()`

Deletes the **first** document matching the filter.

```
db.students.deleteOne({ name: "Dave" })
```

Delete Multiple Documents — `deleteMany()`

Deletes **all** documents matching the filter.

```
// Delete all ECE students
db.students.deleteMany({ branch: "ECE" })

// Delete ALL documents in the collection (collection itself remains)
db.students.deleteMany({})
```

---

12. Finding Documents in MongoDB — Query Operators

MongoDB supports a rich set of **query operators** for filtering documents, equivalent to SQL's WHERE clause operators.

12.1 Comparison Operators

Operator	Meaning	SQL Equivalent
`$eq`	Equal to	`=`
`$ne`	Not equal to	`!=`
`$gt`	Greater than	`>`
`$gte`	Greater than or equal to	`>=`
`$lt`	Less than	`<`
`$lte`	Less than or equal to	`<=`

Operator	Meaning	SQL Equivalent
`$in`	Matches any value in an array	`IN (...)`
`$nin`	Does not match any value in an array	`NOT IN (...)`

```
// Students older than 21
db.students.find({ age: { $gt: 21 } })

// Students in year 1, 2, or 3
db.students.find({ year: { $in: [1, 2, 3] } })

// Students NOT in CS branch
db.students.find({ branch: { $ne: "CS" } })

// Students aged between 20 and 23 (inclusive)
db.students.find({ age: { $gte: 20, $lte: 23 } })
```

12.2 Logical Operators

Operator	Meaning	SQL Equivalent
`$and`	All conditions must be true	`AND`
`$or`	At least one condition must be true	`OR`
`$not`	Negates a condition	`NOT`
`$nor`	None of the conditions must be true	—

```
// CS students in year 2 (AND)
db.students.find({
    $and: [{ branch: "CS" }, { year: 2 }]
})
// Shorthand for AND (both conditions in the same object):
db.students.find({ branch: "CS", year: 2 })

// CS OR IT students (OR)
db.students.find({
    $or: [{ branch: "CS" }, { branch: "IT" }]
})

// Students who are NOT in CS
db.students.find({
    branch: { $not: { $eq: "CS" } }
})
```

12.3 Element Operators

Operator	Meaning
`$exists`	Checks if a field exists in the document
`$type`	Checks if a field is of a specified BSON type

```
// Students who have an email field
db.students.find({ email: { $exists: true } })

// Students who do NOT have an email field
db.students.find({ email: { $exists: false } })
```

12.4 Sorting, Limiting, and Skipping

Sort results (1 = ascending, -1 = descending):

```
// Sort by age ascending
db.students.find().sort({ age: 1 })

// Sort by year descending, then by name ascending
db.students.find().sort({ year: -1, name: 1 })
```

Limit the number of results:

```
// Return only 3 documents
db.students.find().limit(3)
```

Skip a number of results (used for pagination):

```
// Skip the first 5 documents, return the next 5
db.students.find().skip(5).limit(5)
```

12.5 Querying Nested Documents and Arrays

Query a field inside an embedded document (use dot notation):

```
// Students from Mumbai
db.students.find({ "address.city": "Mumbai" })
```

Query for a specific value inside an array:

```
// Students enrolled in DBMS course
db.students.find({ courses: "DBMS" })
```

Query where array contains all specified values:

```
db.students.find({ courses: { $all: ["DBMS", "OS"] } })
```

---

13. Applications of MongoDB

MongoDB is used across a wide range of industries and application types:

13.1 Content Management Systems (CMS)

Blogs, news websites, and publishing platforms store articles, media, tags, and user comments as flexible documents. Since every article may have different metadata (author, tags, categories, embedded images), the schema-less nature of MongoDB is ideal.

13.2 E-Commerce Applications

Product catalogs have highly variable attributes — a shoe has size and color; a laptop has RAM, processor, and storage. Relational tables would require many NULL-filled columns or complex inheritance structures. MongoDB documents naturally accommodate variable product attributes. Order histories and user carts are also well-suited to document storage.

13.3 Real-Time Analytics

MongoDB's aggregation pipeline can process and analyze large volumes of data in real time. It is used for dashboards, monitoring systems, and reporting tools where data arrives continuously and must be queried instantly.

13.4 Mobile Applications

Mobile apps need a backend that can evolve rapidly as the app grows. MongoDB's flexible schema allows developers to add new fields to user profiles, push notifications, or app settings without schema migrations.

13.5 Internet of Things (IoT)

IoT devices generate streams of sensor data (temperature readings, GPS coordinates, device statuses) at very high speed. MongoDB's high write throughput and ability to store time-series-like documents make it a good fit for IoT data storage.

13.6 Social Networks and User Profiles

User profiles on social platforms have complex, variable structures — friends lists, posts, likes, followers, media uploads. MongoDB stores all of this naturally as nested documents and arrays without requiring multiple join tables.

13.7 Gaming

Player profiles, game state, leaderboards, session data, and in-game inventories are stored as documents. Game data changes shape frequently as new features are added — MongoDB's flexibility accommodates this well.

13.8 Logging and Event Tracking

Application logs, server logs, audit trails, and clickstream data are naturally unstructured and arrive at high volume. MongoDB can ingest and store these efficiently, and the aggregation pipeline can analyze patterns across millions of log entries.

---

14. Summary

Concept	Key Point
NoSQL	"Not Only SQL" — non-relational databases for modern, large-scale data
NoSQL Types	Key-Value, Document, Column-Family, Graph, Time Series
SQL vs NoSQL	SQL: structured, ACID, vertical scale; NoSQL: flexible, BASE, horizontal scale
MongoDB	Document-oriented NoSQL database using BSON format
Collection	Equivalent of a SQL table
Document	Equivalent of a SQL row; stores data as JSON/BSON
Field	Equivalent of a SQL column
`_id`	Auto-generated unique identifier; acts as primary key
Embedded Model	Related data stored inside the parent document; fast reads, no joins
Normalized Model	Related data stored in separate collections, linked by `_id` references
`insertOne()` / `insertMany()`	Create documents
`find()` / `findOne()`	Read documents
`updateOne()` / `updateMany()`	Update documents using operators like `$set`, `$inc`
`deleteOne()` / `deleteMany()`	Delete documents
Query Operators	`$gt`, `$lt`, `$in`, `$or`, `$and`, `$exists`, etc.
